You Can Combine All Three Types of Constraints



You can use all of the types of constraints we've seen here—arguments, call counts, and ordering—together in one expectation:

```
expect(catalog).to receive(:search).
  with(/term/).at_least(:twice).ordered
```

Just don't go overboard! Unless all these constraints are actually important, your specs may fail even when the code's external behavior hasn't changed.

Your Turn

In this chapter, we saw several different ways to configure how your test doubles respond to messages. We talked about how to return, raise, or yield specific values. We saw how to replace a method from a real class but still use the original implementation behind the scenes.

We also discussed how to constrain whether a double should respond at all for certain message arguments. By constraining what arguments a message expectation accepts, and how many times it must be called, you can be as specific or as loose as you need in your specs.

Exercises

For these exercises, you're going to dig a little more deeply into the block implementations from *When You Need More Flexibility*, on page 255. Block implementations are the catch-all of RSpec mocks: since you can put *any* arbitrary logic in them, you can always use a block if you can't remember what built-in API does what you need.

Block Implementations

Open exercises/block_implementation_spec.rb in this chapter's source code. It contains seven examples, each of which calls allow(test_double).to receive(:message) with a block.

The first four examples deal with configuring responses via a block:

14-customizing-test-doubles/exercises/block_implementation_spec.rb

```
RSpec.describe "Block implementations that provide responses" do
  let(:test_double) { double }

  it "can return a value" do
    allow(test_double).to receive(:message) do
      # TODO
    end

    expect(test_double.message).to eq(17)
  end
```